Documentación Proyecto Bases 2

Proyecto: restaurant-reservations

Backend analizado a partir de la implementación actual en Express y PostgreSQL.
Se documentan rutas, autenticación, parámetros, cuerpos JSON y respuestas esperadas.

Versión	**1.0.0**
Formato	OpenAPI 3.0.3
Servidor base	http://localhost:3000
Fecha	26 de marzo de 2026
Autores	Dario Soto, Arturo Chavarria

1. Vista general

La API está orientada a la gestión de restaurantes, usuarios, menús, platos, reservas y pedidos. El acceso protegido se maneja con JWT en el header Authorization: Bearer <token>. Para el bootstrap de restaurantes también se admite la clave x-setup-key o el campo setupKey en el cuerpo JSON.

2. Reglas de autenticación y autorización

- Las rutas públicas no requieren token: GET /, GET /restaurants, GET /menus, GET /menus/{id}, GET /menus/{menuId}/dishes y GET /menus/{menuId}/dishes/{id}.
- Las rutas con authenticateToken rechazan solicitudes sin JWT con 401 y tokens inválidos o expirados con 403.
- Algunas rutas exigen además rol admin o pertenecer al restaurante correcto, por ejemplo creación/edición de menús y platos.
- POST /restaurants acepta dos mecanismos: token de un usuario admin o una setup key válida.

3. Códigos de estado usados

Código	Uso principal
200	Consulta o modificación exitosa
201	Recurso creado correctamente
400	Datos faltantes, inválidos o inconsistentes
401	No se envió token o credenciales requeridas
403	Token inválido o falta de permisos
404	Recurso no encontrado
409	Conflicto por duplicidad o estado previo
500	Error interno del servidor

4. Inventario rápido de endpoints

Método	Ruta	Acceso	Descripción
GET	/	Pública	Verifica que la API esté activa
POST	/auth/register	Pública	Registra cliente o admin
POST	/auth/login	Pública	Inicia sesión y devuelve JWT
GET	/restaurants	Pública	Lista restaurantes
POST	/restaurants	JWT admin o setup key	Crea restaurante
GET	/users/me	JWT	Devuelve perfil propio
GET	/users	JWT admin	Lista todos los usuarios
GET	/users/{id}	JWT admin	Busca usuario por ID
PUT	/users/{id}	JWT	Actualiza usuario propio o por admin
DELETE	/users/{id}	JWT admin	Elimina usuario
GET	/menus	Pública	Lista menús; acepta restaurantId

POST	/menus	JWT admin	Crea menú
GET	/menus/{id}	Pública	Consulta un menú
PUT	/menus/{id}	JWT admin dueño	Actualiza menú
DELETE	/menus/{id}	JWT admin dueño	Elimina menú
GET	/menus/{menuId}/dishes	Pública	Lista platos de un menú
POST	/menus/{menuId}/dishes	JWT admin dueño	Crea plato
GET	/menus/{menuId}/dishes/{id}	Pública	Consulta un plato
PUT	/menus/{menuId}/dishes/{id}	JWT admin dueño	Actualiza plato
DELETE	/menus/{menuId}/dishes/{id}	JWT admin dueño	Elimina plato
POST	/reservations	JWT	Crea reserva
DELETE	/reservations/{id}	JWT dueño o admin	Cancela reserva
POST	/orders	JWT	Crea pedido
GET	/orders/{id}	JWT dueño o admin	Obtiene detalle del pedido

5. Detalle por recurso

General

GET /

Acceso: Pública

Descripción: Retorna un mensaje simple para confirmar que el servicio está arriba.

Respuesta exitosa de referencia

```
{"message":"API de Reserva de Restaurantes"}
```

Errores comunes

- 500 - Error interno del servidor

Autenticación

POST /auth/register

Acceso: Pública

Descripción: Crea un usuario. Si role = admin, exige restaurantName y restaurantCode válidos para asociarlo a un local existente.

Campo	Tipo	Req.	Descripción
name	string	Sí	Nombre del usuario
email	string	Sí	Se normaliza a minúsculas
password	string	Sí	Mínimo 8 caracteres
role	string	No	client o admin; por defecto client
restaurantName	string	Condicional	Obligatorio si role = admin
restaurantCode	string	Condicional	Código de seguridad del local

Respuesta exitosa de referencia

```
{
  "message": "Usuario registrado exitosamente",
  "user": {
    "id": 1,
    "name": "Dario",
    "email": "dario@test.com",
    "role": "client",
    "restaurantId": null
  },
  "token": "eyJ..."
}
```

Errores comunes

- 400 - Faltan datos, password corto, rol inválido o email ya registrado
- 401 - Código de seguridad del local incorrecto
- 404 - El local indicado no existe
- 409 - Ese local ya tiene un administrador registrado
- 500 - Error interno del servidor

POST /auth/login

Acceso: Pública

Descripción: Valida credenciales y devuelve JWT con id, email, role y restaurantId.

Campo	Tipo	Req.	Descripción
email	string	Sí	Correo del usuario
password	string	Sí	Contraseña en texto plano

Respuesta exitosa de referencia

```
{
  "message": "Login exitoso",
  "user": {
    "id": 1,
    "name": "Dario",
    "email": "dario@test.com",
    "role": "client",
    "restaurantId": null
  },
  "token": "eyJ..."
}
```

Errores comunes

- 400 - Email y password son obligatorios
- 401 - Credenciales inválidas
- 500 - Error interno del servidor

Restaurantes

GET /restaurants

Acceso: Pública

Descripción: Lista id, nombre y fecha de creación de los restaurantes.

Respuesta exitosa de referencia

```
{
  "message": "Restaurantes obtenidos correctamente",
  "restaurants": [
    { "id": 1, "name": "Soda TEC", "created_at": "2026-03-25T00:00:00.000Z" }
  ]
}
```

Errores comunes

- 500 - Error interno del servidor

POST /restaurants

Acceso: JWT admin o x-setup-key

Descripción: Crea un restaurante nuevo. Se autoriza con token admin o con setup key enviada por header o en el body.

Header alternativo: x-setup-key: clave-setup

Campo	Tipo	Req.	Descripción
name	string	Sí	Nombre del local
adminCode	string	Sí	Código de seguridad interno del local
setupKey	string	No	Alternativa al header x-setup-key

Respuesta exitosa de referencia

```
{
 "message": "Restaurante creado exitosamente",
 "restaurant": {
  "id": 3,
  "name": "Nuevo Local",
  "created_at": "2026-03-25T00:00:00.000Z"
 }
}
```

Errores comunes

- 400 - Nombre del local y código son obligatorios o código muy corto
- 403 - No autorizado para crear restaurantes
- 409 - Ya existe un restaurante con ese nombre
- 500 - Error interno del servidor

Usuarios

GET /users/me

Acceso: JWT

Descripción: Devuelve el perfil del usuario autenticado sin password_hash.

Respuesta exitosa de referencia

```
{
 "message": "Perfil de usuario",
 "user": {
  "id": 1,
  "name": "Dario Soto",
  "email": "dario@test.com",
  "role": "client",
  "restaurant_id": null,
  "created_at": "2026-03-25T00:00:00.000Z",
  "updated_at": "2026-03-25T00:00:00.000Z"
 }
}
```

Errores comunes

- 401 - Token no proporcionado
- 403 - Token inválido o expirado
- 404 - Usuario no encontrado
- 500 - Error interno del servidor

GET /users

Acceso: JWT admin

Descripción: Lista todos los usuarios registrados.

Respuesta exitosa de referencia

{"message":"Usuarios obtenidos","users":[{"id":1,"name":"Dario","email":"dario@test.com","role":"client","restaurant_id":null}]}

Errores comunes

- 401 - No autenticado
- 403 - No tienes permisos para acceder a este recurso
- 500 - Error interno del servidor

GET /users/{id}

Acceso: JWT admin

Descripción: Obtiene un usuario específico por su ID.

Nombre	Ubicación	Tipo	Req.	Descripción
id	path	integer	Sí	ID del usuario

Respuesta exitosa de referencia

{"message":"Usuario encontrado","user":{"id":1,"name":"Dario","email":"dario@test.com","role":"client","restaurant_id":null}}

Errores comunes

- 401 - No autenticado
- 403 - No tienes permisos para acceder a este recurso
- 404 - Usuario no encontrado
- 500 - Error interno del servidor

PUT /users/{id}

Acceso: JWT propio usuario o admin

Descripción: Actualiza datos del usuario. Un admin puede además modificar role y restaurantId.

Nombre	Ubicación	Tipo	Req.	Descripción
id	path	integer	Sí	ID del usuario

Campo	Tipo	Req.	Descripción
name	string	No	Nuevo nombre
email	string	No	Debe ser único
password	string	No	Se almacena hasheado
role	string	No	Solo aplica cuando el solicitante es admin
restaurantId	integer	No	Solo aplica cuando el solicitante es admin

Respuesta exitosa de referencia

{"message":"Usuario actualizado","user":{"id":1,"name":"Dario Soto","email":"nuevo@test.com","role":"client","restaurant_id":null}}

Errores comunes

- 400 - Email ya en uso o no hay campos para actualizar
- 403 - No tienes permiso para modificar este usuario
- 404 - Usuario no encontrado
- 500 - Error interno del servidor

DELETE /users/{id}

Acceso: JWT admin

Descripción: Elimina un usuario por ID.

Nombre	Ubicación	Tipo	Req.	Descripción
id	path	integer	Sí	ID del usuario

Respuesta exitosa de referencia

{"message":"Usuario eliminado correctamente"}

Errores comunes

- 403 - No tienes permiso para eliminar usuarios
- 404 - Usuario no encontrado
- 500 - Error interno del servidor

Menús

GET /menus

Acceso: Pública

Descripción: Lista todos los menús o filtra por restaurantId si se envía en query.

Nombre	Ubicación	Tipo	Req.	Descripción
restaurantId	query	integer	No	Filtra por restaurante

Respuesta exitosa de referencia

{"message":"Menús obtenidos correctamente","menus":[{"id":5,"name":"Ejecutivo","description":"Menú del día","restaurant_id":10,"restaurant_name":"Pizza CR"}]}

Errores comunes

- 500 - Error interno del servidor

POST /menus

Acceso: JWT admin del restaurante

Descripción: Crea un menú. Si restaurantId no se envía, usa el restaurantId del token del admin.

Campo	Tipo	Req.	Descripción

name	string	Sí	Nombre del menú
description	string	No	Descripción libre
restaurantId	integer	No	Debe coincidir con el restaurante del admin

Respuesta exitosa de referencia

{"message":"Menú creado exitosamente","menu":{"id":5,"name":"Ejecutivo","description":"Menú del día","restaurant_id":10}}

Errores comunes

- 400 - El nombre del menú es obligatorio
- 403 - No autorizado para crear menús o intento de usar otro restaurante
- 500 - Error interno del servidor

GET /menus/{id}

Acceso: Pública

Descripción: Obtiene un menú por ID.

Nombre	Ubicación	Tipo	Req.	Descripción
id	path	integer	Sí	ID del menú

Respuesta exitosa de referencia

{"message":"Menú encontrado","menu":{"id":5,"name":"Ejecutivo","description":"Menú del día","restaurant_id":10,"restaurant_name":"Pizza CR"}}

Errores comunes

- 404 - Menú no encontrado
- 500 - Error interno del servidor

PUT /menus/{id}

Acceso: JWT admin dueño

Descripción: Actualiza un menú. Solo puede hacerlo el admin del restaurante propietario.

Nombre	Ubicación	Tipo	Req.	Descripción
id	path	integer	Sí	ID del menú

Campo	Tipo	Req.	Descripción
name	string	No	Nuevo nombre
description	string	No	Nueva descripción; puede enviarse vacío

Respuesta exitosa de referencia

{"message":"Menú actualizado","menu":{"id":5,"name":"Nuevo menú","description":"Descripción actualizada","restaurant_id":10}}

Errores comunes

- 400 - No hay campos para actualizar

- 403 - No tienes permiso para modificar este menú
- 404 - Menú no encontrado
- 500 - Error interno del servidor

DELETE /menus/{id}

Acceso: JWT admin dueño

Descripción: Elimina un menú existente.

Nombre	Ubicación	Tipo	Req.	Descripción
id	path	integer	Sí	ID del menú

Respuesta exitosa de referencia


```
{"message":"Menú eliminado correctamente"}
```


Errores comunes

- 403 - No tienes permiso para eliminar este menú
- 404 - Menú no encontrado
- 500 - Error interno del servidor

Platos

GET /menus/{menuId}/dishes

Acceso: Pública

Descripción: Lista todos los platos pertenecientes a un menú específico.

Nombre	Ubicación	Tipo	Req.	Descripción
menuId	path	integer	Sí	ID del menú

Respuesta exitosa de referencia


```
{"message":"Platos obtenidos","dishes":[{"id":10,"name":"Pizza","description":"Grande","price":"3500.00","menu_id":5}]}
```


Errores comunes

- 404 - Menú no encontrado
- 500 - Error interno del servidor

POST /menus/{menuId}/dishes

Acceso: JWT admin dueño

Descripción: Crea un plato dentro del menú indicado.

Nombre	Ubicación	Tipo	Req.	Descripción
menuId	path	integer	Sí	ID del menú

Campo	Tipo	Req.	Descripción

name	string	Sí	Nombre del plato
description	string	No	Descripción del plato
price	number	Sí	Precio numérico

Respuesta exitosa de referencia

{"message":"Plato creado","dish":{"id":10,"name":"Pizza Suprema","description":"Extra queso","price":3500,"menu_id":5}}

Errores comunes

- 400 - Nombre y precio son obligatorios
- 403 - No autorizado para agregar platos a este menú
- 404 - Menú no encontrado
- 500 - Error interno del servidor

GET /menus/{menuId}/dishes/{id}

Acceso: Pública

Descripción: Consulta un plato por su ID.

Nombre	Ubicación	Tipo	Req.	Descripción
menuId	path	integer	Sí	ID del menú
id	path	integer	Sí	ID del plato

Respuesta exitosa de referencia

{"message":"Plato encontrado","dish":{"id":10,"name":"Pizza","description":"Grande","price":"3500.00","menu_id":5}}

Errores comunes

- 404 - Plato no encontrado
- 500 - Error interno del servidor

PUT /menus/{menuId}/dishes/{id}

Acceso: JWT admin dueño

Descripción: Actualiza un plato existente.

Nombre	Ubicación	Tipo	Req.	Descripción
menuId	path	integer	Sí	ID del menú
id	path	integer	Sí	ID del plato

Campo	Tipo	Req.	Descripción
name	string	No	Nuevo nombre
description	string	No	Nueva descripción
price	number	No	Nuevo precio

Respuesta exitosa de referencia

{"message":"Plato actualizado","dish":{"id":10,"name":"Pizza XL","description":"Nueva versión","price":4200,"menu_id":5}}

Errores comunes

- 400 - No hay campos para actualizar
- 403 - No tienes permiso para modificar este plato
- 404 - Plato no encontrado
- 500 - Error interno del servidor

DELETE /menus/{menuId}/dishes/{id}

Acceso: JWT admin dueño

Descripción: Elimina un plato existente.

Nombre	Ubicación	Tipo	Req.	Descripción
menuId	path	integer	Sí	ID del menú
id	path	integer	Sí	ID del plato

Respuesta exitosa de referencia

{"message":"Plato eliminado correctamente"}

Errores comunes

- 403 - No tienes permiso para eliminar este plato
- 404 - Plato no encontrado
- 500 - Error interno del servidor

Reservas

POST /reservations

Acceso: JWT

Descripción: Crea una reserva asociada al usuario autenticado.

Campo	Tipo	Req.	Descripción
restaurantId	integer	Sí	ID del restaurante
date	string	Sí	Fecha en formato YYYY-MM-DD
time	string	Sí	Hora, por ejemplo 18:00
numPeople	integer	Sí	Cantidad de personas

Respuesta exitosa de referencia

{"message":"Reserva creada","reservation":{"id":3,"user_id":7,"restaurant_id":1,"reservation_date":"2026-03-30","reservation_time":"18:00","num_people":2,"status":"pending"}}

Errores comunes

- 400 - Faltan datos obligatorios
- 401 - Token no proporcionado

- 403 - Token inválido o expirado
- 500 - Error interno del servidor

DELETE /reservations/{id}

Acceso: JWT dueño o admin

Descripción: Cancela una reserva existente.

Nombre	Ubicación	Tipo	Req.	Descripción
id	path	integer	Sí	ID de la reserva

Respuesta exitosa de referencia


```
{"message":"Reserva cancelada","reservation":{"id":2,"user_id":7,"status":"cancelled"}}
```


Errores comunes

- 403 - No autorizado
- 404 - Reserva no encontrada
- 500 - Error interno del servidor

Pedidos

POST /orders

Acceso: JWT

Descripción: Crea un pedido asociado al usuario autenticado y luego agrega sus ítems.

Campo	Tipo	Req.	Descripción
restaurantId	integer	Sí	ID del restaurante
orderType	string	No	pickup o dine_in; por defecto pickup
items	array	Sí	Lista de elementos del pedido
items[].dishId	integer	Sí	ID del plato
items[].quantity	integer	Sí	Cantidad pedida

Respuesta exitosa de referencia


```
{
 "message": "Pedido creado",
 "order": {
  "id": 77,
  "user_id": 4,
  "restaurant_id": 1,
  "order_type": "pickup",
  "status": "pending",
  "total": "7000.00"
 }
}
```


Errores comunes

- 400 - Datos incompletos o plato no encontrado

- 404 - Restaurante no encontrado
- 500 - Error interno del servidor

GET /orders/{id}

Acceso: JWT dueño o admin

Descripción: Devuelve el pedido y sus ítems.

Nombre	Ubicación	Tipo	Req.	Descripción
id	path	integer	Sí	ID del pedido

Respuesta exitosa de referencia

```
{
 "message": "Detalles del pedido",
 "order": {
   "id": 88,
   "user_id": 4,
   "restaurant_id": 1,
   "total": "7000.00"
 },
 "items": [
   { "id": 1, "dish_id": 10, "quantity": 2, "subtotal": "7000.00" }
 ]
}
```

Errores comunes

- 403 - No autorizado
- 404 - Pedido no encontrado
- 500 - Error interno del servidor

6. Archivos generados

Además de este documento, se generaron los siguientes archivos listos para usar:

- openapi_restaurante.yaml - especificación OpenAPI principal para Swagger Editor o Swagger UI.
- openapi_restaurante.json - misma especificación en formato JSON.

7. Observaciones finales

- La documentación está alineada con las rutas realmente montadas en src/app.js.
- Las reglas de permisos fueron tomadas de los controladores y middlewares actuales.
- Las respuestas de ejemplo siguen los nombres de campo tal como salen del backend, por eso en algunos casos se usan restaurantId y en otros restaurant_id.